

For Windows OS

1. Create the main project folder

Open **Command Prompt** or **PowerShell**:

```
mkdir PHP-Lab  
cd PHP-Lab
```

Initialize Git:

```
git init
```

Now your folder is a Git repository.

2. Create multiple folders

For example, create folders for different weeks:

```
mkdir Week-01-Basics  
mkdir Week-02-Variables  
mkdir Week-03-If-Else  
mkdir Week-04-Loops  
mkdir Week-05-Switch-Case  
mkdir Week-06-Combined
```

Your structure will be:

```
PHP-Lab  
|  
|— Week-01-Basics  
|— Week-02-Variables  
|— Week-03-If-Else  
|— Week-04-Loops  
|— Week-05-Switch-Case
```

└── Week-06-Combined

3. Create files inside folders

In Windows CMD:

```
type nul > Week-01-Basics\task1.php
```

```
type nul > Week-01-Basics\task2.php
```

```
type nul > Week-02-Variables\task1.php
```

```
type nul > Week-02-Variables\task2.php
```

```
type nul > Week-03-If-Else\task1.php
```

```
type nul > Week-03-If-Else\task2.php
```

```
type nul > Week-04-Loops\task1.php
```

```
type nul > Week-04-Loops\task2.php
```

```
type nul > Week-05-Switch-Case\task1.php
```

```
type nul > Week-05-Switch-Case\task2.php
```

```
type nul > Week-06-Combined\cafeteria.php
```

Check your files:

```
dir /s
```

4. Check Git status

```
git status
```

You should see your PHP files as **untracked files**.

5. Add all folders and files

You don't need to add every folder separately.

Simply use:

```
git add .
```

This means:

Add all new and modified files inside the current repository.

Check again:

```
git status
```

6. Make your first commit

```
git commit -m "Add PHP lab tasks"
```

Now your files are stored in Git's history.

7. Create GitHub repository

Go to GitHub and create a new repository, for example:

PHP-Lab

Don't create another README if you already created one locally, to keep the first push simple.

Then copy your GitHub repository URL.

For example:

<https://github.com/username/PHP-Lab.git>

Add GitHub as the remote:

`git remote add origin https://github.com/username/PHP-Lab.git`

Check it:

`git remote -v`

8. Rename your branch to main

`git branch -M main`

Then push:

`git push -u origin main`

Now your complete folder structure will be available on GitHub.

9. Adding a new folder later

Suppose next week you want:

Week-07-Functions

Create it:

```
mkdir Week-07-Functions
```

Create a PHP file:

```
type nul > Week-07-Functions\functions.php
```

Then check:

```
git status
```

Add it:

```
git add .
```

Commit:

```
git commit -m "Add Week 7 functions lab"
```

Push:

```
git push
```

That's it.

10. Adding files to only one folder

Suppose you modify:

```
Week-04-Loops\task1.php
```

You can add only that file:

```
git add Week-04-Loops\task1.php
```

Then:

```
git commit -m "Update loop task"  
git push
```

You don't have to commit the entire repository.

11. Adding an entire folder only

For example:

```
git add Week-05-Switch-Case
```

Then:

```
git commit -m "Add switch case tasks"  
git push
```

12. See your commit history

```
git log
```

A shorter version:

```
git log --oneline
```

Example:

```
a81f32d Add Week 6 combined task  
b72d891 Add Week 5 switch case tasks  
c51a920 Add Week 4 loop tasks
```

13. Check which remote repository is connected

```
git remote -v
```

Example:

```
origin https://github.com/username/PHP-Lab.git (fetch)
```

```
origin https://github.com/username/PHP-Lab.git (push)
```

14. Get the latest changes from GitHub

If you are working from another computer:

```
git pull
```

Or explicitly:

```
git pull origin main
```

15. Clone the complete repository to another Windows computer

You don't need to create all the folders manually.

Just:

git clone <https://github.com/username/PHP-Lab.git>

Then:

cd PHP-Lab

All folders will automatically appear:

PHP-Lab

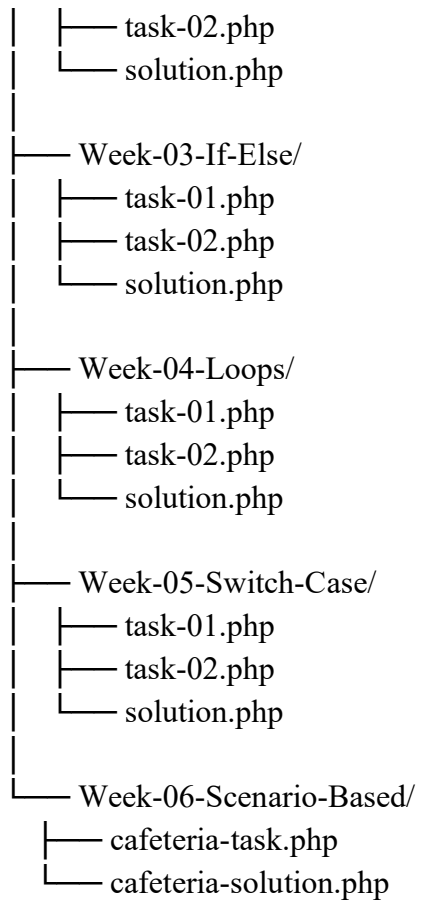
```
|
|— Week-01-Basics
|— Week-02-Variables
|— Week-03-If-Else
|— Week-04-Loops
|— Week-05-Switch-Case
|— Week-06-Combined
```

16. Recommended structure for your PHP lab

For your teaching/lab work, I'd recommend:

PHP-Lab/

```
|
|— README.md
|
|— Week-01-PHP-Basics/
|   |— task-01.php
|   |— task-02.php
|   |— solution.php
|
|— Week-02-Variables/
|   |— task-01.php
```



Complete command sequence

If you want the **whole process from zero**, you can use:

```
mkdir PHP-Lab
```

```
cd PHP-Lab
```

```
git init
```

```
mkdir Week-01-PHP-Basics
```

```
mkdir Week-02-Variables
```

```
mkdir Week-03-If-Else
```

```
mkdir Week-04-Loops
```

```
mkdir Week-05-Switch-Case
```

```
mkdir Week-06-Scenario-Based
```

```
type nul > Week-01-PHP-Basics\task-01.php
```

```
type nul > Week-02-Variables\task-01.php
type nul > Week-03-If-Else\task-01.php
type nul > Week-04-Loops\task-01.php
type nul > Week-05-Switch-Case\task-01.php
type nul > Week-06-Scenario-Based\cafeteria-task.php
```

```
git add .
git commit -m "Add PHP lab structure and tasks"
```

```
git branch -M main
```

```
git remote add origin https://github.com/USERNAME/PHP-Lab.git
```

```
git push -u origin main
```

The basic cycle you'll use every week

After the repository is already connected to GitHub, you only need:

```
git status
git add .
git commit -m "Add Week 7 lab"
git push
```

So the workflow is:

Create/edit files → git add . → git commit → git push

And you can keep adding **unlimited folders inside the same repository**.